

Inheritance and Polymorphism

Programming and Data Structures — Week 1, Lecture 3

Dr. Innocent Nyalala

The repetition problem

```
class Circle:
    def __init__(self, r):
        self.r = r
    def area(self):
        return 3.14159 * self.r ** 2

class Square:
    def __init__(self, s):
        self.s = s
    def area(self):
        return self.s ** 2
```

Nothing is wrong yet, but if you later want “loop over a list of shapes and print each area,” you need code that treats a Circle and a Square the same way despite them being unrelated classes.

Inheritance: naming the relationship

```
1 class Shape:
2     def area(self):
3         raise NotImplementedError # forces every subclass to override this
4
5 class Circle(Shape):                # Circle IS-A Shape
6     def __init__(self, r):
7         self.r = r
8     def area(self):
9         return 3.14159 * self.r ** 2
```

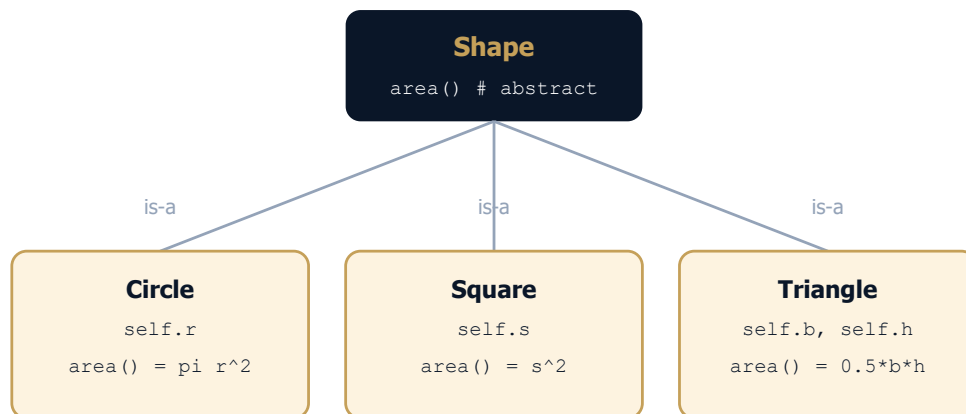
```

10
11 class Square(Shape):                # Square IS-A Shape
12     def __init__(self, s):
13         self.s = s
14     def area(self):
15         return self.s ** 2

```

`class Circle(Shape):` means “Circle inherits from Shape.” `Shape.area` exists only to guarantee every subclass provides its own — calling it directly on a bare `Shape` is a deliberate crash, catching the mistake of forgetting to override it.

The hierarchy, drawn out



Same method name, `area()`, different formula per subclass — that is polymorphism.

`Triangle(Shape)` fits the same pattern: its own `__init__`, its own `area()` formula, the same `Shape` parent.

Polymorphism: one call, many behaviors

```

shapes = [Circle(2), Square(3), Triangle(4, 5)]
for s in shapes:
    print(type(s).__name__, s.area())

```

```

Circle 12.566
Square 9
Triangle 10.0

```

The loop body never checks “is this a Circle or a Square?” It calls `s.area()` and trusts each object to answer for its own type. This is **polymorphism**: the same method call, `area()`, produces different behavior depending on which object it is called on. New shape added later? The loop above needs zero changes.

Where this pays off later in the course

A `Stack` and a `LinkedListStack` can both implement `push` and `pop`. Code that uses a stack — say, a parenthesis checker — can be written once against “anything with `push` and `pop`” and swapped between implementations without being rewritten. Polymorphism is what makes that swap free.

MTech addition — composition vs. inheritance

```
class Engine:
    def start(self):
        return "engine started"

class Car:
    def __init__(self):
        self.engine = Engine()    # composition: Car HAS an Engine

    def start(self):
        return self.engine.start()
```

Inheritance answers “*is this a kind of that?*” Composition answers “*does this have one of those?*” Confusing the two produces hierarchies that do not make sense. A `Car` is **not** a kind of `Engine`; it *contains* one. `class Car(Engine)` would technically run, but `car.start()` would then mean “the car itself starts,” collapsing a real-world distinction the code needs to keep.

MTech addition — duck typing and ABCs

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self): ...

Shape()          # TypeError: can't instantiate abstract class Shape
```

Python does not require a shared base class to treat two objects identically — if an object has `.area()`, code that calls `.area()` accepts it, base class or not. `abc.ABC` is how you turn that implicit expectation into an enforced one. Without ABC, forgetting to override `area()` on a subclass fails only when `area()` is finally called, possibly far from the class definition. With ABC, it fails immediately, at the point the mistake was made.

Exercise

Add `Triangle(Shape)` to the hierarchy above, with `__init__(self, b, h)` and `area()` returning $0.5 * b * h$. Confirm it prints correctly inside the `for s in shapes:` loop with no changes to that loop.

Before next lecture

We now have a working `Shape` family. Next lecture: why every data structure in the rest of this course — starting with `Stack` next week — is built using exactly this pattern, and why counting operations (not seconds) is how we will judge them.